TABLETUNE: MEASURED, SELF-CORRECTING DRAFT-LENGTH TABLES FOR DYNAMIC SPECULATIVE DECODING

Anonymous Authors[1]

ABSTRACT

Speculative decoding speeds up large language model (LLM) serving only while the GPU has spare compute, so the best *draft length*, the number of tokens a small drafter guesses per step, shrinks as the batch grows. vLLM lets operators set the draft length for each batch size with a hand-written table, but nothing measures or corrects that table. We present TableTune. Its *tuner* times the decoding step for every batch size and draft length on vLLM's own serving path and computes, in minutes, the table that maximizes throughput. Its *controller*, a vLLM plugin, tracks how many guesses are accepted in live traffic and rebuilds the table while serving. We measure TableTune only against other ways to speculate. On an H100, the tuned table delivers at least 94% of the throughput of the best fixed draft length at every load in four of five drafter settings (88% in the fifth) without the operator having to know which length that is. Against reimplementations of TurboSpec- and Nightjar-style control on the same engine, its worst-case loss is never higher than theirs, and with the EAGLE-3 drafter it is 3% against 9% and 12%. On a mixture-of-experts model, vLLM's documented table loses up to 18% and the tuned table at most 3%. Starting from mistuned tables, the controller raises throughput by up to 58% (up to 45% on vLLM's default runner), and after a workload switch with the DFlash drafter it recovers 96% of a stale table's loss. In llm-d's optimized-baseline deployment as shipped, two Qwen3-32B replicas on four GPUs, the tuned table is up to 32% faster than a fixed draft length of 3 and up to 25% faster than vLLM's documented table at high load, and the controller raises the documented table's throughput by up to 29%.

1 INTRODUCTION

Generating text with an LLM is slow because it is sequential: each new token requires a full pass through the model, and each pass mostly waits on reading the model's weights from GPU memory (Pope et al., 2023). *Speculative decoding* (Leviathan et al., 2023; Chen et al., 2023) hides part of this cost. A small, fast *drafter* guesses the next few tokens; the *target model* checks all guesses in one pass, keeps those it agrees with, and adds one token of its own. Because every kept token is verified, the output is exactly what the target would have produced alone.

Speculation is not free. Every guess is extra work, and wrong guesses are thrown away. On an idle GPU the extra work is nearly free, because the GPU waits on memory anyway; on a busy GPU serving many requests it slows every step, and speculation can make serving slower than not speculating (Liu et al., 2024b). The best number of guesses per step, the *draft length*, therefore depends on load.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

vLLM (Kwon et al., 2023) recently added *Dynamic Speculative Decoding* (Dynamic SD) (Ranjan, 2026), which lets the draft length depend on the *batch size*, the number of requests in a step. The operator supplies a table such as "batch sizes 1–64: guess 3 tokens; 65–128: guess 1; above: none." The table is written by hand, often copied from the documentation, and nothing checks or corrects it. Users report throughput collapsing when the batch oscillates around a table edge (vLLM contributors, 2026a) and Dynamic SD running 21–31% slower than no speculation on an H100 (vLLM contributors, 2026b).

TableTune. TableTune keeps vLLM's mechanism and makes its table measured and self-correcting (Figure 2). Its **tuner** runs once per GPU, engine, model and drafter: it times one decoding step for a grid of batch sizes and every draft length through vLLM's own serving path, measures how often guesses are accepted, and computes the table that maximizes useful tokens per second at every batch size. Its **controller**, a vLLM plugin that needs no engine changes, re-estimates acceptance from live traffic and rebuilds the table while serving. Both emit vLLM's own table format. Prior work established the ingredients: TurboSpec (Liu et al., 2024b) chooses the draft length that maximizes goodput from profiled step times, Su et al. (2023) make it depend

on the batch size, and Nightjar (Li et al., 2025a) learns it online. TurboSpec and Nightjar change the engine's decisions; TableTune instead makes vLLM's existing production table measured and self-correcting, with step times measured on the serving path rather than fitted, a controller that runs as a plugin, and a three-GPU study of what that table can deliver and what it needs from the engine.

Contributions.

1. *A tuner that measures the table* (Sections 4.1, 6.2). On an H100 with vLLM's V2 runner, which keeps full CUDA graphs for every draft length (Section 2), at every load, the tuned table delivers at least 94% of the throughput of the best fixed draft length for that load with the EAGLE-3 drafter (code prompts, chat prompts, a 4-bit target) and with n-gram drafting, and at least 88% with DFlash, where any single fixed draft length drops to 81% or less at some load. Under *jumping load*, which changes every 50 seconds, it delivers in every 50-second window at least 98.5% of the throughput of the fastest alternative (fixed draft length 1 or 2, or vLLM's documented table). On a mixture-of-experts target the best length stays long at every load, and the tuned table stays within 2.9% where vLLM's documented table, written with dense models in mind, loses up to 17.5%. On a T4 and an A10G, for full-precision targets, the tables deliver at least 94% of it, where every fixed draft length drops 7.6–43.6% below it at some load.
2. *A comparison with prior adaptive control* (Section 6.3): on the same engine and lengths, reimplementations of TurboSpec-style per-step control lose up to 9.3% with EAGLE-3, because a fitted step-time model misses a jump in step time, and of Nightjar-style bandits up to 11.8%, because every batch size keeps exploring; the tuned table loses at most 3.3%.
3. *A controller that repairs tables while serving* (Sections 4.2, 6.4): up to +58% on the H100 from a mistuned table, +23% for DFlash started from another drafter's table, and 96% of a stale table's 10.8% loss after a workload switch with DFlash on the H100 (72–85% on the A10G).
4. *End-to-end gains in llm-d* (Section 6.5), a Kubernetes-native distributed serving stack built on vLLM (llm-d contributors, 2026): on its optimized-baseline guide as shipped (Qwen3-32B with tensor parallelism over two GPUs, TP=2), without changes to llm-d or vLLM, up to +32% throughput over fixed length 3 and +25% over vLLM's documented table at high load, and up to +29% from the controller started from that table; with four Qwen3-8B replicas, up to +22% and +40%.
5. *What table-based speculation needs from the engine* (Section 6.6): full CUDA graphs per draft length (vLLM's default V1 runner loses 30–43% on the H100 with any table, regardless of host cores), a real off switch, and length switches that do not stall the engine.

2 BACKGROUND

Batches and decoding. A serving engine runs one forward pass over a *batch* of requests per *step*; the *batch size* B is the number of requests in it. With *continuous batching* (Yu et al., 2022) requests join and leave every step, so B follows the traffic. Our benchmarks set the *concurrency* c, the number of open requests. The live batch, vLLM's count of running requests, is smaller: on average about half to two thirds of c in our closed-loop runs. Tables are indexed by the live batch; our figures plot c. In a decode step the GPU reads every weight once, so with few tokens per step the step time is set by memory bandwidth and extra tokens cost little, up to the *ridge point* (Williams et al., 2009) (about 200 tokens per step on a T4, 210 on an A10G and 300 on an H100 SXM).

Speculative decoding. A drafter proposes k tokens. The target scores them in one pass and accepts guesses up to the first disagreement, then appends one token of its own (Leviathan et al., 2023; Chen et al., 2023). With a_i the probability that guess i and all before it are accepted, the expected *tokens per step* is $\tau(k) = 1 + \sum_{i=1}^{k} a_i$. EAGLE-style drafters (Li et al., 2024b;c; 2025b) are small heads on the target's features; a *draft model* is a separate small LLM; *n-gram* drafting (Saxena, 2023) copies prompt text; DFlash (Chen et al., 2026) drafts a block of tokens in one pass of a small diffusion model.

Goodput. We maximize *goodput*, kept tokens per second (Liu et al., 2024b): the tuner predicts it with Equation 1, and we measure it as output tokens per second (throughput). With step time $T(B, k)$,

$$G(B, k) = B \cdot \tau(k) \,/\, T(B, k). \tag{1}$$

Speculation raises the numerator and the denominator; whether it pays depends on which grows faster at that B.

vLLM Dynamic SD and CUDA graphs. vLLM takes a table `num_speculative_tokens_per_batch_size` of ranges $[B_{\text{start}}, B_{\text{end}}, k]$ and looks up k for each step (Ranjan, 2026); the documentation's example is $[[1, 64, 3], [65, 128, 1], [129, 512, 0]]$. A *CUDA graph* (NVIDIA, 2025b) replays a recorded sequence of kernels with one launch, saving CPU launch time. With

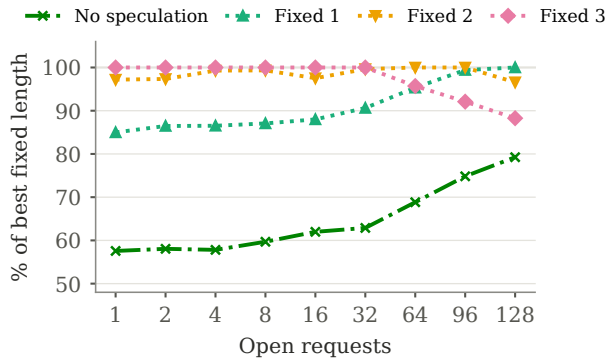


Figure 1. No fixed draft length is best at every load. H100 (V2 runner), Qwen3-4B + EAGLE-3, code prompts at temperature 1.0, mean of three runs: throughput as % of the best fixed draft length at each load.

a table, vLLM's default V1 model runner replaces its *full* decode graphs with *piecewise* graphs, because the tokens per request now vary; its V2 runner keeps full graphs per draft length since October 2026 (vLLM contributors, 2026c).

3 Motivation

Figure 1 shows the problem on an H100 with Qwen3-4B (Qwen Team, 2025) and its EAGLE-3 drafter (Li et al., 2025b). *The best length changes with load:* length 3 is best at low load and length 1 at 128 requests; fixed length 1 loses 15% at 1 request and length 3 loses 12% at 128. *The best table changes with the GPU and the engine:* the tuned tables are $[[1, 18, 3], [19, 43, 2], [44, 128, 1]]$ on a T4, $[[1, 16, 3], [17, 60, 2], [61, 91, 1], [92, 128, 0]]$ on an A10G and $[[1, 50, 3], [51, 128, 2]]$ on the H100 with the V2 runner; the H100's plain step grows only from 4.4 to 6.1 ms between a batch of 1 and 128, the T4's from 35.7 to 92.3 ms. *The engine decides whether any table can pay:* on the default V1 runner a table costs about 40% of the throughput at low load (Table 1). *Acceptance changes with the traffic:* EAGLE-3's tokens per step at length 3 range from 1.81 (translation) to 2.43 (math) across seven Spec-Bench (Xia et al., 2024) and code workloads, on every GPU we measured.

These observations suggest a split. Step time depends on hardware, engine and batch size, is stable, and can be measured once offline. Acceptance depends on traffic, drifts, and should be tracked online. TableTune follows this split.

4 Design

4.1 The tuner

Step time. Let K be the longest draft length the server may use, vLLM's `num_speculative_tokens` (3 for EAGLE-3, 5 for n-gram and the draft model, 8 for DFlash). For each draft length k from 0 to K the tuner starts vLLM with the real drafter and a one-range table $[[1, B_{\max}, k]]$, so the timed path is exactly the serving path, including the graph mode the server will use. For each batch size B it sends B prompts of the workload's typical length (32 tokens by default) twice, asking once for 1 output token and once for 65; the difference is 64 decode steps, so $T(B, k) = (t_{65} - t_1)/64$. It keeps the median of three repeats, over every batch size up to 16 and then in steps of 4 to 8 up to $B_{\max}$. To keep the batch fixed while timing, it uses vLLM's test mode that rejects every guess: each step does the full drafting and verification work of length k, but all B requests advance one token per step and finish together. The benefit of accepted guesses is added back through $\tau(k)$. Work that grows with the number of accepted tokens is not timed this way; Section 6.7 measures what that misses.

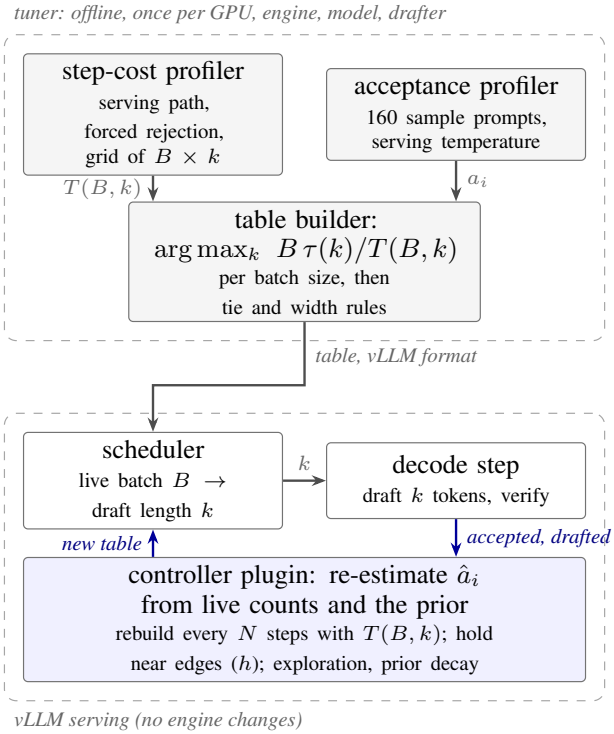


Figure 2. TableTune. The tuner measures step time on vLLM's serving path and acceptance on sample prompts, and writes a table in vLLM's own format. While serving, the controller plugin reads vLLM's acceptance counters, re-estimates acceptance and rebuilds the table with the tuner's step times.

Acceptance. The tuner then measures how often guesses are accepted. It runs the drafter at the longest length, K guesses per step, on 160 sample prompts, at the temperature the server will use, and reads vLLM's own published speculative-decoding metrics, `vllm:spec_decode_num_drafts` and `vllm:spec_decode_num_accepted_tokens_per_pos`: how many drafts were made and, for every position i, how many of them kept guess i. Their ratio is a_i. One run covers every shorter length too, because a draft of length k is just the first k guesses of a draft of length K.

The table. For each batch size B the tuner computes the goodput of every draft length (Equation 1) and picks the highest. Two rules keep the table short. It keeps the previous batch size's length unless another is more than 0.5% better, so near-ties do not create edges, and it merges any range narrower than 25% of its first batch size into a neighbour. Some drafters guess only on some steps: n-gram drafts only when the text it has seen contains a match. For them the tuner multiplies acceptance by the share of steps with a guess, the *proposal rate*, but still charges the full step time $T(B,k)$, since a step in which some requests draft costs about as much as one in which all do. From the same profiles, a GPU-free *screening* predicts each fixed length's worst loss, so an operator can keep a fixed length when a table would not help.

4.2 The controller

The controller is a vLLM general plugin that answers the scheduler's draft-length lookup. It learns acceptance from live traffic and turns it into a table.

What it observes. The controller adds no instrumentation of its own. After every step it receives the counts that vLLM's scheduler records for those same published metrics: for each request, how many guesses were drafted (d) and how many were accepted (m; verification stops at the first wrong guess, so the accepted guesses are always the first m). The controller keeps two counts per draft position i: drafted_i, the number of drafts that reached position i ($d \geq i$), and accepted_i, the number in which guess i was accepted ($m \geq i$). A request that drafts 3 guesses and keeps 2 adds one to $\text{drafted}_{1..3}$ and to $\text{accepted}_{1..2}$. The ratio $\text{accepted}_i/\text{drafted}_i$ estimates a_i, the probability that the first i guesses are all accepted.

From acceptance to draft length. Acceptance sets what each extra guess is worth: since $\tau(k) = 1 + \sum_{i \leq k} a_i$, going from length $k-1$ to k gains a_k tokens per step and costs $T(B,k) - T(B,k-1)$. The gain falls with k because a_k falls, and the cost grows with B, so for each B the best length is where the next guess stops paying, the maximum of Equation 1. Higher acceptance therefore keeps long drafts up to larger batches; lower acceptance moves the table's edges to smaller batches and shorter lengths, down to length 0. The step times never change while serving, so acceptance is the only input the controller updates.

Estimating and rebuilding. Every N steps it computes $\hat{a}_i = (w\, a_i^{\text{prior}} + \text{accepted}_i)/(w + \text{drafted}_i)$, a blend of the tuner's values (worth $w = 2000$ observations) and the live counts, which fade by half after each rebuild so recent traffic dominates, and rebuilds the table from Equation 1 with the tuner's step times. A position that is never drafted keeps its prior: while the table runs length 1 or 0 the controller learns nothing about later guesses. Two options matter when the traffic changes: *exploration* (one step at the longest allowed length every 50 steps, so every position keeps being observed) and *prior decay* (the prior's weight also halves at each rebuild). Near a table edge it holds the current length while the batch size is within h of the edge (*hysteresis*), avoiding the oscillation of (vLLM contributors, 2026a). By default it uses only the lengths in its starting table, which have profiles and captured graph sizes; under V1 an opt-in mode allows any profiled length, while under V2 only the starting table's lengths have full CUDA graphs.

5 Implementation

TableTune is about 1,880 lines of Python with 104 test functions. The tuner is one command (`dsd-tuner run`). Because the table is a vLLM argument and the controller an installed plugin enabled by environment variables, TableTune needs no engine changes and deploys into serving stacks such as llm-d by configuration alone. The tuner runs unchanged on vLLM's V1 and V2 runners. Code, per-GPU tables, benchmark scripts, raw results and analysis scripts are available as an anonymized artifact.[1]

6 Evaluation

We answer six questions. *Q1* (Section 6.2): does a measured table hold up against every fixed draft length, across loads, drafters and GPUs? *Q2* (Section 6.3): how does it compare with prior adaptive controllers? *Q3* (Section 6.4): does the controller repair mistuned tables and follow a workload change? *Q4* (Section 6.5): do the gains carry over to a production stack like llm-d? *Q5* (Section 6.6): what must the engine provide? *Q6* (Section 6.7): what does TableTune cost, and how reliably can step times be measured?

[1] Anonymized link provided to reviewers on OpenReview.

6.1 Setup

Hardware and software. Our main platform is an H100 80 GB SXM (3.35 TB/s, BF16) on an OpenShift cluster, a whole GPU with a 16-CPU quota on a Xeon 8480+ host. We ran the main matrix of tuner, sweep and controller experiments twice on it: with vLLM `353e491ef` and its default V1 runner, and with a vLLM nightly (`43b4aaea`, October 2026) and the V2 runner, which keeps full CUDA graphs per draft length. Because a table cannot pay under V1 (Section 6.6), all later studies use V2 only. The repository lists every study with its scripts, tables and raw results. For comparison we ran the same experiments, with the same scripts, prompts and loads and vLLM's V1 runner, on a Tesla T4 (FP16) and an A10G (BF16), both AWS instances with 2-core hosts. Targets are Qwen3-4B, its 4-bit AWQ version (Lin et al., 2024), the mixture-of-experts Qwen3-30B-A3B (Qwen Team, 2025) and, in llm-d, Qwen3-32B at TP=2 and Qwen3-8B; drafters are EAGLE-3, n-gram, Qwen3-0.6B as a draft model, and DFlash.

Workloads and load. AIPerf (NVIDIA, 2025a) keeps c requests open in a closed loop; a *sweep* measures every load c from 1 to 128, with $4c$ requests per level after c warm-up requests, and the maximum batch is 128. The main workload is *code*: InstructCoder (Li et al., 2024a) instructions (about 32 prompt tokens), answers up to 256 tokens, temperature 1.0. n-gram uses *code-apply* (an instruction plus the code to edit, about 151 tokens in, answers up to 512 tokens, greedy); the chat study uses Spec-Bench (greedy, up to 64 requests).

Isolation and statistics. Each H100 V1 study ran alone on an otherwise idle node, because piecewise CUDA graphs turned out to be sensitive to host contention (Section 6.6); the V2 studies ran on nodes shared with other teams' GPU jobs, which full graphs are insensitive to ($\pm$4%). We report output tokens per second per GPU, the mean of three runs unless marked; the run-to-run spread on the H100 is 1.4% (median) and 5.7% (95th percentile) with V2 and 2.5%/11.2% with V1, and at most 4.1% and 6.5% on the T4 and A10G. We treat gaps below about 2% as ties. Unless stated otherwise the controller uses hysteresis $h = 8$ and rebuilds the table every 200 steps; the workload-switch runs use its defaults ($h = 4$, every 500 steps) with exploration every 50 steps and prior decay 0.5. Draft lengths and acceptance reported in the evaluation come from the same vLLM metrics, read from each server's `/metrics` endpoint, so every number can be checked against vLLM's own accounting.

Baselines. *Best fixed* at a load is the fastest fixed draft length there, without a table and with full graphs, chosen after the fact; losses are against it. *Best single fixed length* is the one length with the smallest worst-case loss over all loads, also chosen after the fact: an operator would have to benchmark every length to find it. We credit TableTune only with differences from such speculative configurations (and from vLLM's documented table and the adaptive baselines of Section 6.3); what speculation gains over decoding without it is the drafter's, not the tuner's, and we show it only for context.

6.2 Q1: measured tables against fixed lengths

Table 1 is the main result. For each GPU it compares the tuned table's worst loss with that of the one fixed length an operator would have to find by benchmarking every length.

On the H100 with V2, the tuned tables stay within 3.0–5.7% of the best fixed length at every load for EAGLE-3, the 4-bit target, n-gram and Spec-Bench, and within 11.6% for DFlash, whose best single fixed length loses 19.1%. Here a well-chosen single length is nearly as safe: 3.4% for EAGLE-3, and for n-gram and Spec-Bench the tuner itself returns one length, the one an exhaustive benchmark picks (5 and 3). So on one H100 up to 128 requests the table's value is that it finds that length without benchmarking every length. Where no single length works, the table keeps finding the right one: beyond 128 requests, where in llm-d fixed length 3 falls up to 24% behind it (Section 6.5); on a mixture-of-experts target, with DFlash, and on the older GPUs (below).

A mixture-of-experts target behaves differently. With Qwen3-30B-A3B (30B parameters, 3B active per token) and its EAGLE-3 drafter on one H100 with V2, the step without speculation grows from 4.7 ms at a batch of 1 to 19.7 ms at 128, because a larger batch activates more experts, while three draft tokens add 61% at a batch of 1 but only 31% at 128, once those experts are loaded. Speculation therefore keeps paying as load grows: length 3 is the fastest fixed length at every load from 2 to 96 requests and length 2 at 128 (by 2.3%), and the tuner finds that table ($[[1, 3, 3], [4, 4, 2], [5, 128, 3]]$), within 2.9% of the best fixed length at every load. A table written for a dense model does not fit here: vLLM's documented table switches to length 1 above 64 requests and loses 17.5% at 96 and 12.2% at 128 requests (three runs).

Long prompts change the table less. With GovReport summarization prompts (Huang et al., 2021) cut to 1,024 and 3,072 tokens (EAGLE-3, three runs), a longer prompt makes every step costlier, and the tuner re-profiled at that length moves the table's edge from a batch of 50 to 36–38. Its tables stay within 0.7% and 1.5% of the best fixed length at every load, where fixed length 1 loses up to 16%; but the 32-token table also stays within 2.0%, and vLLM's documented table within 2.3%: on the H100 the step grows

Table 1. Worst loss (%) against the best fixed draft length at each load, over 1–128 open requests (Spec-Bench: 1–64), mean of three runs. *Table* is TableTune's tuned table; *Best fixed* is the one fixed draft length with the smallest worst-case loss (length in brackets), which an operator could only find by benchmarking every length. Lower is better; bold marks the better of the two. H100 V2: full CUDA graphs per draft length; H100 V1: vLLM's default runner, which switches to piecewise CUDA graphs whenever a table is set. Older GPUs: the same settings from our T4 and A10G study, each run on the GPU named (EAGLE-3 code on both). †single run. ‡the *0/1/3 table*, built from the tuner's length-0, 1 and 3 profiles only (the *all-length table*, built from every profiled length, steps from length 2 to 1, which stalls this vLLM nightly; Section 6.6). §the tuned table is that single fixed length, so the two differ only by run-to-run noise. n/a: the tuner's V2 profile of the draft model is invalid (Section 6.7).

	H100 V2		H100 V1		Older GPUs		
Setting	Table	Best fixed	Table	Best fixed	GPU	Table	Best fixed
EAGLE-3, code	4.0	**3.4** (2)	39.5	**3.1** (2)	T4 / A10G	**4.0 / 2.3**	9.6 (2) / 15.0 (1)
EAGLE-3, 4-bit AWQ target	**3.0**‡	4.6 (2)	43.0	**4.2** (2)	T4	20.6†	**9.2** (1)†
n-gram, code-apply	5.7§	4.5 (5)	5.2	**0.0** (5)	T4	**2.8**	7.6 (5)
Qwen3-0.6B draft model, code	n/a	0.0 (1)	35.6	**0.0** (1)	T4	**3.5**†	12.2 (1)†
DFlash, code	**11.6**	19.1 (3)	29.5	**20.4** (3)	A10G	**5.2**	18.0 (3)
EAGLE-3, Spec-Bench chat	3.1§	3.9 (3)	39.3	**2.5** (2)	T4	3.8	**2.2** (2)

slowly enough that prompt length barely matters.

On the older GPUs, for full-precision targets on the code workloads, the tables stay within 2.3–5.2% where the best single fixed length loses 7.6–18.0%, and every fixed length loses 7.6–43.6% at some load.

On the H100 with the default V1 runner, the same kind of table loses 30–43% for every model-based drafter (n-gram, which adds no model pass, 5.2%): the cost of the engine's graph mode, not of the tuner's choices (Section 6.6).

Jumping load. We drove one server per configuration through eleven 50-second windows whose open requests jump between 2 and 128 (the A10G's schedule, three runs; two client processes on the H100, since one cannot keep up with no speculation). On the A10G the tuned table never fell below 81% of the best configuration in a window (mean 94.0%). On the H100 with V2 it was the best configuration in practically every window: **98.5–100%** of the per-window best (mean 99.6%), 4.1% above fixed length 1, 1.1% above the documented table and level with fixed length 2 (-0.2%). With V1, fixed length 2 beat it by 15%.

Open-loop arrivals. We also sent requests at 2 to 72 per second with Poisson and with bursty arrivals (gamma inter-arrival times, smoothness 0.25), on the H100 with V2, three runs each. Below saturation every configuration keeps up, so the draft length shows in latency, and no single fixed length is right at every rate. At 2 requests per second the median request takes 0.63–0.64 s with the tuned table and with fixed length 3, and 18% longer with fixed length 1; at 72 per second the table takes 1.08 s and fixed length 1 1.10 s, while fixed length 3, which keeps length 3 at batch sizes where it no longer pays, is 22–24% slower. Bursty arrivals change none of these orderings.

6.3 Q2: against prior adaptive control

Baselines. TurboSpec and Nightjar released no code, so we reimplemented each as a vLLM plugin following its paper, in its own package that shares no code with TableTune. *TurboSpec-style* picks the length every step, for the whole batch, maximizing goodput from a linear step-time model (a target pass over $B(k{+}1)$ tokens plus k drafting passes) fitted to the tuner's profile at eight batch sizes, and from one per-token acceptance rate, a moving average over steps, with the geometric formula for tokens per step (Liu et al., 2024b). *Nightjar-style* runs Nightjar's ADA-BINGREEDY bandit for every batch size, with the measured time per output token as its loss (Li et al., 2025a); offloading the drafter to CPU memory needs engine changes and is omitted. Both start from the tuner's acceptance profile where they use one, and, like TableTune's controller, may only choose lengths that are in their starting table, the only ones with full CUDA graphs under V2.

Results. With EAGLE-3 (Figure 3a) the tuned tables stay within 3.3% (all-length table) and 3.7% (0/1/3 table) of the best fixed length at every load, 3.0% with the controller, TurboSpec-style within 9.3% and Nightjar-style within 11.8%. TurboSpec-style matches the tables up to 32 requests and then keeps length 3 too long: its linear model ranks length 3 first up to a batch of about 100, while the measured profile switches to length 2 above 50, because the fit smooths over the jump in length-3 step time (at a batch of 80, measured 9.7 ms, fitted 8.0 ms). The same choice makes its requests 20–23% slower than with the table at 72 open-loop requests per second. Nightjar-style loses 4–12% at every load because each batch size learns separately and keeps exploring. With DFlash (Figure 3b) all three trail fixed length 3 by 10–29% above 32 requests, a length none of them has: the tuned table uses 2/4/6/8 and predicted length 4 to beat 3 there, which the measured sweep contra-

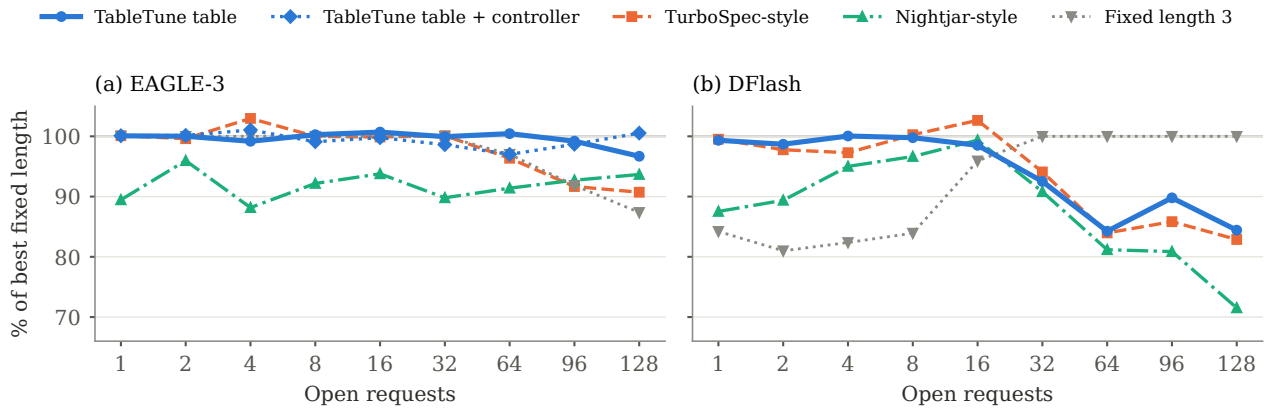


Figure 3. TableTune against TurboSpec-style and Nightjar-style control on the H100 (V2 runner), code prompts at temperature 1.0, mean of three runs. Throughput as % of the best fixed draft length at each load; every method chooses among the lengths of its starting table: with EAGLE-3, 0/1/3 for TurboSpec- and Nightjar-style, 2/3 for the all-length tuned table and 1/3 for the 0/1/3 table (also the controller's start); with DFlash, 2/4/6/8 for all.

dicts (Section 6.7 explains why); among the three, TableTune loses least (15.8%, against 17.1% and 28.5%). In a repeat of the jumping-load study that adds the two baselines (greedy, three runs), the tuned tables never fall below 95% of the best configuration in a window (the all-length table: 98%; 98.8–99.2% on average); TurboSpec-style and Nightjar-style fall to 89% and 92% in the high-load windows and deliver 5.3% and 4.1% fewer tokens in total than the all-length table. With greedy acceptance fixed length 2 is nearly as good as the tables here (98.3% on average).

6.4 Q3: the controller

Repairing mistuned tables. A table is *mistuned* when its ranges or lengths do not fit the GPU, drafter or workload it serves. The most natural case is vLLM's documented table, deployed as an operator copies it. In llm-d with Qwen3-32B it drops to length 0 above a batch of 128 per replica, and started from it the controller raises throughput by 9–29% at 256–1024 open requests, to the tuned table's level (Section 6.5); on one H100 up to 128 requests the same table is nearly right, and the controller adds only 2%. To test repair under larger errors, Table 2 and Figure 4 also start it from deliberately mistuned tables, with an acceptance prior measured on a different workload. From the documented table with its ranges divided by 4, which drops to length 0 above a batch of 32, it gains up to 58% on the H100 (V2) and 45% (V1), against 36% on the T4 and 14% on the A10G: the H100's wider gap between speculation and none makes a mistuned table costlier and its repair larger. At every load and on every GPU it reaches 96–103% of the tuned table's throughput (Figure 4a, circles). On the mixture-of-experts target the same mistuned table costs far more, because speculation pays at every load there: the controller raises throughput by 71–83% at 40–96 requests (three runs), again to the tuned table's level. Started from another drafter's table it rebuilds the right one: +23% for DFlash on the H100 with V2, reaching the tuned table's throughput with only the starting table's lengths 1–3 (V2 has full graphs only for those), and +37% under V1, where it may use any profiled length. At loads where the starting table is already near-optimal it changes throughput by −3.7 to +3%. Under V1 on the H100's 4-bit target it costs 9–24% (single run), consistent with its Python work on a launch-bound step; under V2 the cost disappears.

A workload switch. We switched the prompts mid-run between Spec-Bench math and translation, the pair whose tables differ most, at the load where they disagree (80 open requests on the A10G and the H100 with V2, 71 with V1); after the switch the first workload's table is *stale*. On the A10G, where translation's table runs length 0 and math's length 2 there, the stale table loses 7.5–18.1%; the controller with its defaults oscillates or stays at length 0, where it observes nothing, while with exploration and prior decay it recovers 72% and 85% of the gap and rebuilds math's table exactly. We therefore recommend both options as defaults. On the H100 the same EAGLE-3 switch leaves little to recover (−4.5 to 1.5%, Table 2): EAGLE-3's acceptance varies too little across workloads for its H100 tables to differ much. DFlash's varies far more (5.03 tokens per step at length 8 on math, 2.51 on translation), so we repeated the switch with DFlash, code then math at 96 open requests, where code's table runs length 2 and math's length 6 (Figure 4b). The stale code table loses 10.8% after the switch (three runs); the controller rebuilds math's table within the first minute and recovers 96% of the gap (+11.7%, within 0.4% of math's own table), and before the switch it beats code's own table by 1.5% by learning that length 4 pays at this load. TurboSpec-style also recovers it (+12.4%), since it too tracks acceptance online; Nightjar-style recov-

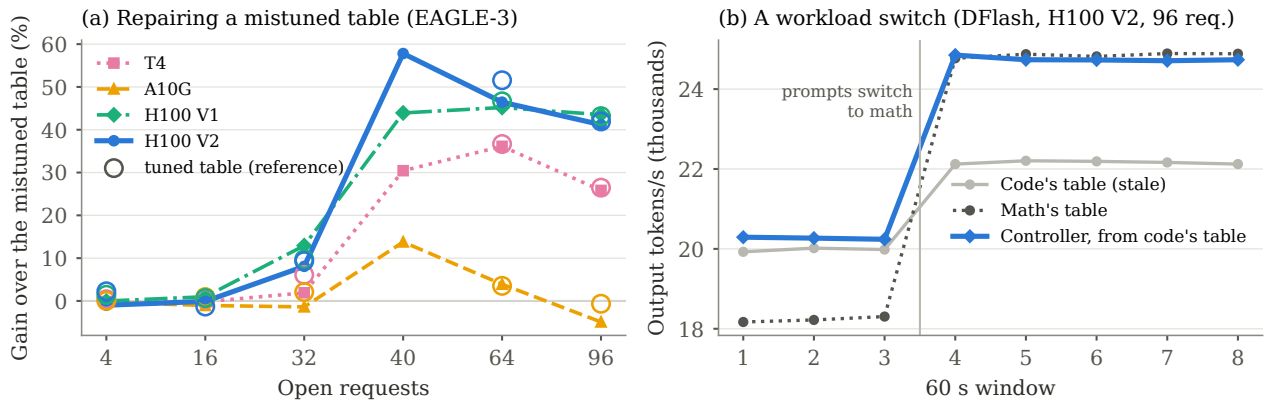


Figure 4. The controller. (a) Repairing the same mistuned table on three GPUs: vLLM's documented table with its ranges divided by 4 (length 0 above a batch of 32), started with a mismatched chat prior, EAGLE-3 on code. Lines: gain of the controller over the same table without it; hollow circles: gain of that GPU's tuned table, the reference the controller should reach (no sweep point at 40). (b) A workload switch: DFlash on the H100 (V2), 96 open requests, code prompts, then math from the fourth 60-second window. Mean of three runs.

Table 2. The controller: best gain over the same table without it, at the load where it occurs (loads 4–96; n-gram 4–128), mean of three runs unless marked (†single run). Top: steady traffic, starting from a mistuned table. Bottom: a mid-run workload switch (Spec-Bench math and translation with EAGLE-3; DFlash: code then math at 96 requests; greedy, A10G protocol), the stale table's loss and what the controller with exploration and prior decay does about it. Older GPUs: the same experiment from our T4 and A10G study, on the GPU named. §still 60% below no speculation, which beats every draft length of this drafter under V2.

Starting table	H100 V2	H100 V1	Older GPUs
Documented table / 4 (Figure 4a)	**+58%** (40)	+45% (64)	T4 +36% (64), A10G +14% (40)
Documented table	+2% (16)	+8% (96)	T4 +15% (64), A10G +16% (64)
n-gram, hand-written table	**+30%** (4)†	+22% (4)†	T4 +24% (16)
Draft model, EAGLE-3's table	+40% (96)†§	+2%†	T4 +1%†
DFlash, EAGLE-3's table	+23% (64)	+37% (64)	A10G +11% (4)
4-bit AWQ, documented table	+5% (64)†	−9 to −24%†	T4 +23% (64)†
Workload switch: stale table's loss	−4.5 to 0.4%; DFlash 10.8%	1.0–1.5%	A10G 7.5–18.1%
controller (both options)	−4.4 to +1.4%; DFlash recovers 96%	−2.5 to +3.2%	A10G recovers 72–85%

ers 38%, because its bandit has to relearn every batch size. From translation to math at 48 requests the tables differ (length 4 against 8) but their throughput does not (−1.1%), and every controller stays within 0.6% of math's own table.

6.5 Q4: end to end in llm-d

Setup. llm-d's optimized-baseline guide (llm-d contributors, 2026) serves vLLM replicas behind the llm-d router, an Envoy proxy with an endpoint picker that routes by prefix-cache affinity and token load. We deployed the guide as shipped, two replicas of Qwen3-32B at TP=2 on four H100s with the guide's Helm values for the router, which are calibrated for this setup, and changed only configuration: a Kustomize overlay on the guide's model-server base adds the model's EAGLE-3 drafter and the tuned table to `--speculative-config`, uses a vLLM nightly image with the V2 runner, caps each replica at 256 sequences of 4096 tokens, and for the controller puts the TableTune package on `PYTHONPATH` and sets its `VLLM_DSD_*` variables; the overlay is in the repository. The tuner profiled one replica at TP=2 with the same image in about 30 minutes; its all-length table steps from length 2 to 1, which stalls this nightly (Section 6.6), so we deploy its 0/1/3 table, $[[1, 60, 3], [61, 256, 1]]$. As a second scale we ran four Qwen3-8B replicas at TP=1 on the same GPUs. AIPerf sent code prompts through the router at 4–1024 open requests, three runs per configuration (run-to-run spread 3.2% median).

Results. On the guide as shipped (Figure 5a), up to 128 open requests the tuned table runs length 3 and matches fixed length 3 and the documented table within run-to-run noise. From 256 the replicas cross the table's edge to length 1, and there it is 20–32% faster than fixed length 3 and 7–25% faster than the documented table, which drops to length 0 above a batch of 128 per replica; at 512 and 1024 open requests the median time between tokens is 23–29% lower than with either, and the median first token

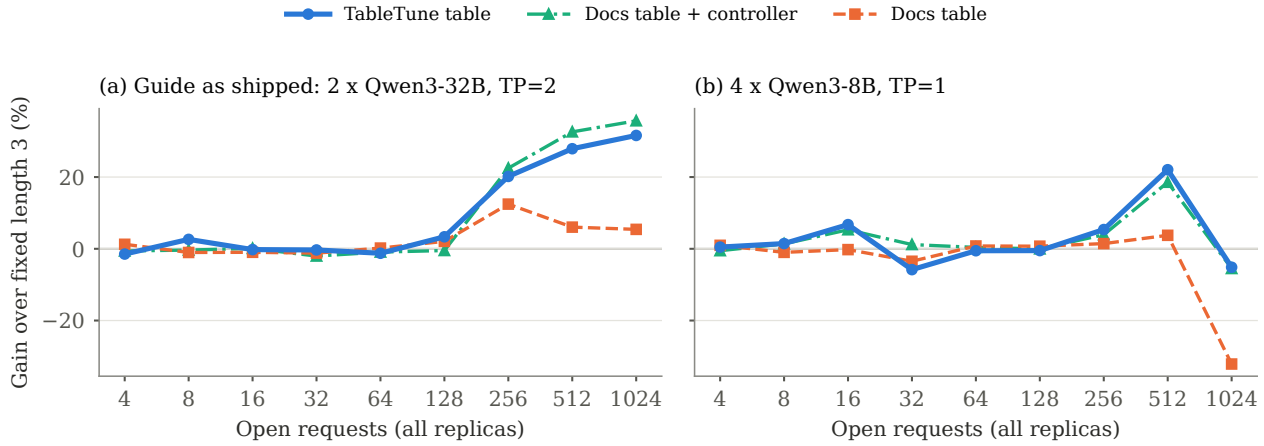


Figure 5. llm-d optimized-baseline with EAGLE-3 behind the llm-d router, code prompts at temperature 1.0, mean of three runs: output tokens/s against fixed draft length 3 (0 = fixed length 3). (a) The guide's own model and layout, 2 × Qwen3-32B at TP=2 on four H100s. (b) 4 × Qwen3-8B at TP=1 on the same GPUs; the 32-request dip is shared by every speculative configuration. The tuned table with the controller is within 2% of the table alone up to 512 requests in (b), and up to 4% above it in (a); it is omitted.

arrives 23–26% sooner at 1024. (Speculation itself, with any of these configurations, adds 48–78% over the guide as shipped at up to 64 requests; that gain belongs to the drafter.) Started from the documented table, the controller raised throughput by 9–29% at 256–1024 open requests, to the tuned table's level, and on top of the tuned table it added up to 4%. With four Qwen3-8B replicas (Figure 5b) the table is up to 22% faster than fixed length 3 (5% behind it at 1024) and up to 40% faster than the documented table, and the controller adds 14% and 39% to the latter at 512 and 1024 requests.

6.6 Q5: what the engine must provide

Full CUDA graphs per draft length. On vLLM's default V1 runner any table switches the decode step to piecewise CUDA graphs, which add CPU launch work to every step (a Nsight trace on the T4 shows the GPU busy 91% of the time with full graphs and 60% with piecewise ones). A long step hides that work, as on the older GPUs; the H100's step is short (about 6 ms for Qwen3-4B), so it shows: piecewise graphs alone cost fixed length 3 24–34% at 1–32 requests, more host cores do not help (2, 4 or 16), and host contention makes it worse. With V2's full graphs per draft length the loss disappears (Table 1). Until an engine provides them, keep a table only if it beats the best fixed length and no speculation, both timed with full graphs.

Length switches that do not stall. On this vLLM nightly, several tables that step from length 2 to length 1 stalled the V2 runner under load (the 4-bit target's tuned table, EAGLE-3's T4 table, translation's table, and llm-d's at its 80/81 edge). The 0/1/3 tables, and tables without such a step, never stalled; we report those where marked, and the fault upstream.

A real off switch. A table entry of 0 still runs the drafter, because EAGLE-style drafters keep their own KV cache. Where no speculation is best (the A10G above 96 requests), the tuned table trails it by 5.6–6.7%, and on the H100 the A10G's table, at length 0 from 92 requests, runs 19–22% below no speculation. Nightjar (Li et al., 2025a) provides one.

6.7 Q6: cost and reliability of measuring

Overheads. Profiling takes about 8 minutes on the H100, 20 for the mixture-of-experts target, 28 for Qwen3-32B at TP=2, 22 on the A10G and 45 on the T4, once per GPU, engine, model and drafter; a new workload needs only an acceptance profile of about 2 minutes. The controller adds no GPU work and rebuilds the table every N steps in the scheduler process.

Measure, do not fit. Step times must be measured per GPU and engine. On the H100 a linear model misses a jump in length-3 step time and costs TurboSpec-style control 9% (Section 6.3); on the A10G every simple model we fitted misses the switch to length 0 and loses 12.7% at 128 requests.

Where the profile misses: DFlash. Timing with every guess rejected does not time work that grows with accepted tokens. For DFlash that matters: its acceptance in the sweep matches the profile, but at length 8 its steps take 1.3–1.9× the profiled time at live batches of 6–38, against 1.05–1.16× at length 1. Profiling with the measured acceptance instead (vLLM's synthetic rejection sampler, steps counted from its draft counter) corrects the step times but barely changes the table: in both profiles length 3 wins only in short ranges above a batch of about 47, which the

25% minimum-width rule merges into length 4. A tolerance repairs this: merge a narrow range only if that loses less than 2% of its predicted goodput, our tie threshold. It changes four of our V2 tables; rerun against the original ones in the same session (three runs), it makes DFlash 6.5% faster at 64 requests, n-gram about 5% faster at 96–128 and Spec-Bench 3% faster at 64, and leaves EAGLE-3 within noise; the other tables do not change. It is a tuner option (`-merge-tol`); the results above use the original rule. DFlash still trails fixed length 3 by 10% at 128 requests, serving work such as interleaved prefill that a decode-only profile cannot see.

Near-ties and an invalid profile. With EAGLE-3 on the H100, lengths 2 and 3 nearly tie, and two profiles of the same GPU differed by 4–9% in single step times, enough to move an edge at a cost of a few percent. Under V2 the profile of the Qwen3-0.6B draft model is flat at about 40 ms, against 9–37 ms when served, so that table is invalid.

7 Limitations

Engines. The results with full graphs use one vLLM nightly, whose V2 runner stalls when a table steps from length 2 to 1 (Section 6.6); where that matters, including both llm-d deployments, we report the 0/1/3 table, which can be slightly worse than the all-length table.

Accuracy of the table. The tuner's choices are only as good as its profiles. With DFlash the table loses up to 16% to fixed length 3 at batches of 34–87; a merge tolerance repairs part of it, but serving work the profile cannot see leaves a 10% gap at 128 requests (Section 6.7), and the controller, whose prior was accurate, does not correct cost errors. The draft model's V2 profile was invalid, and its T4 results at high load are decided by KV-cache pressure and preemption, which the step-time profile cannot see.

Scope. Dense targets of 4B, 8B and 32B parameters and one mixture-of-experts target with 3B active parameters, on H100s in a single node at TP of at most 2; prompts of 32 tokens in most studies and of up to 3,072 tokens in one, answers up to 256 tokens (512 for n-gram); closed-loop load, plus open-loop arrivals up to 72 requests per second on one GPU. Prompts beyond 4,096 tokens, mixed and recorded traffic, larger mixture-of-experts models, multi-node and disaggregated serving are missing. TurboSpec and Nightjar released no code, so we compare with reimplementations that follow their papers on unmodified vLLM, without Nightjar's drafter offloading and TurboSpec's context-length term. Some H100 nodes ran other teams' jobs; we compare runs only within a node and session.

8 Related Work

Drafters and per-request adaptation. Drafter designs (Li et al., 2024b;c; 2025b; Cai et al., 2024; Miao et al., 2024; Chen et al., 2024; Saxena, 2023; Fu et al., 2024; Chen et al., 2026) and methods that adapt the length per request or token (Mamou et al., 2024; Huang et al., 2025a; Agrawal et al., 2024; Hou et al., 2025; Liu et al., 2025; Zhang et al., 2025) or update the drafter online (Liu et al., 2024a) are complementary to choosing the length from load.

Load-aware speculation in serving. Su et al. (Su et al., 2023) proposed choosing the speculation length per batch size, the idea behind vLLM's Dynamic SD table. BASS (Qian et al., 2024) batches speculation, TETRIS (Wu et al., 2025) picks which draft tokens to verify across a batch, and Sequoia (Chen et al., 2024) sizes its token tree for the hardware. TurboSpec (Liu et al., 2024b) is closest to our work: it maximizes goodput inside vLLM with a linear step-time model fitted to profiled runs and a moving average of recent acceptance, choosing the length at each step. TableTune applies these ideas to vLLM's production table, measures the grid instead of fitting it, and keeps the table current with a plugin; Section 6.3 compares it with reimplementations of TurboSpec and Nightjar. Nightjar (Li et al., 2025a) picks the length per batch size online with a multi-armed bandit, switches speculation truly off and moves the drafter to CPU memory. SpecServe/AdaSpec (Huang et al., 2025b) and AdaServe (Li et al., 2026) adapt speculation to latency targets; we optimize throughput. MagicDec (Sadhukhan et al., 2025) finds speculation pays at large batches for long contexts, and Kong et al. (Kong et al., 2026) model its latency under load.

9 Conclusion

A draft-length table must be measured for each GPU, engine and model and kept current as traffic changes; TableTune measures it in minutes and repairs it while serving. On an H100 with full CUDA graphs its tables stay within 3–6% of the best fixed length for most drafters and deliver at least 98.5% of the best configuration in every window under jumping load, with lower worst-case loss than reimplemented TurboSpec- and Nightjar-style control. Its controller recovers up to 58% from mistuned tables, and in llm-d's optimized-baseline guide as shipped the tuned table is up to 32% faster than a fixed draft length and 25% faster than vLLM's documented table.

Tables do not transfer: on a mixture-of-experts model a table written for a dense model loses up to 18%. On vLLM's default runner the same tables lose 30–43% to the graph mode, so engines should provide full graphs per draft length, stall-free length switches and a real off switch.

REFERENCES

Agrawal, S., Jeon, W., and Lee, M. AdaEDL: Early draft stopping for speculative decoding of large language models via an entropy-based lower bound on token acceptance probability. In *NeurIPS Workshop on Efficient Natural Language and Signal Processing*, 2024.

Cai, T., Li, Y., Geng, Z., Peng, H., Lee, J. D., Chen, D., and Dao, T. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Chen, C., Borgeaud, S., Irving, G., Lespiau, J.-B., Sifre, L., and Jumper, J. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Chen, J., Liang, Y., and Liu, Z. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.

Chen, Z., May, A., Svirschevski, R., Huang, Y., Ryabinin, M., Jia, Z., and Chen, B. Sequoia: Scalable and robust speculative decoding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

Fu, Y., Bailis, P., Stoica, I., and Zhang, H. Break the sequential dependency of LLM inference using lookahead decoding. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Hou, Y., Zhang, F., Du, C., Zhang, X., Pan, J., Pang, T., Du, C., Tan, V. Y. F., and Yang, Z. BanditSpec: Adaptive speculative decoding via bandit algorithms. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025.

Huang, K., Guo, X., and Wang, M. SpecDec++: Boosting speculative decoding via adaptive candidate lengths. In *Conference on Language Modeling (COLM)*, 2025a.

Huang, K., Wu, H., Shi, Z., Zou, H., Yu, M., and Shi, Q. AdaSpec: Adaptive speculative decoding for fast, SLO-aware large language model serving. In *ACM Symposium on Cloud Computing (SoCC)*, 2025b. First released as SpecServe, arXiv:2503.05096.

Huang, L., Cao, S., Parulian, N., Ji, H., and Wang, L. Efficient attentions for long document summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 2021.

Kong, L., Flynn, M., Peng, M., Shavit, N., Kurtz, M., and Marques, A. An interpretable latency model for speculative decoding in LLM serving. *arXiv preprint arXiv:2605.15051*, 2026.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

Li, K., Hu, Q., Zhao, X., Chen, H., Xie, Y., Liu, T., Xie, Q., and He, J. InstructCoder: Instruction tuning large language models for code editing. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Student Research Workshop)*, 2024a.

Li, R., Zhang, Z., Zhang, L., Wang, H., Fu, X., and Lai, Z. Nightjar: Dynamic adaptive speculative decoding for large language models serving. *arXiv preprint arXiv:2512.22420*, 2025a.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024b.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024c.

Li, Y., Wei, F., Zhang, C., and Zhang, H. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025b.

Li, Z., Chen, Z., Delacourt, R., Oliaro, G., Wang, Z., Chen, Q., Lin, S., Yang, A., Zhang, Z., Chen, Z., Lai, Y.-H., Cheng, X., Miao, X., and Jia, Z. AdaServe: Accelerating multi-SLO LLM serving with SLO-customized speculative decoding. In *European Conference on Computer Systems (EuroSys)*, 2026.

Lin, J., Tang, J., Tang, H., Yang, S., Chen, W.-M., Wang, W.-C., Xiao, G., Dang, X., Gan, C., and Han, S. AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024.

Liu, T., Li, Y., Lv, Q., Liu, K., Zhu, J., Hu, W., and Sun, X. PEARL: Parallel speculative decoding with adaptive draft length. In *International Conference on Learning Representations (ICLR)*, 2025.

Liu, X., Hu, L., Bailis, P., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. Online speculative decoding. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024a.

Liu, X., Park, J., Hu, L., Kwon, W., Li, Z., Zhang, C., Du, K., Mo, X., You, K., Cheung, A., Deng, Z., Stoica, I., and Zhang, H. TurboSpec: Closed-loop speculation control system for optimizing LLM serving goodput. *arXiv preprint arXiv:2406.14066*, 2024b. First released as "Optimizing Speculative Decoding for Serving Large Language Models Using Goodput".

llm-d contributors. llm-d: Kubernetes-native distributed inference serving, optimized-baseline guide. https://github.com/llm-d/llm-d/tree/main/guides/optimized-baseline, 2026. Commit 4dc1573, October 2026.

Mamou, J., Pereg, O., Korat, D., Berchansky, M., Timor, N., Wasserblat, M., and Schwartz, R. Dynamic speculation lookahead accelerates speculative decoding of large language models. *arXiv preprint arXiv:2405.04304*, 2024.

Miao, X., Oliaro, G., Zhang, Z., Cheng, X., Wang, Z., Zhang, Z., Wong, R. Y. Y., Zhu, A., Yang, L., Shi, X., Shi, C., Chen, Z., Arfeen, D., Abhyankar, R., and Jia, Z. SpecInfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024.

NVIDIA. AIPerf: Benchmarking tool for generative AI inference. https://github.com/ai-dynamo/aiperf, 2025a.

NVIDIA. CUDA C++ Programming Guide: CUDA graphs. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2025b.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems (MLSys)*, 2023.

Qian, H., Gonugondla, S. K., Ha, S., Shang, M., Gouda, S. K., Nallapati, R., Sengupta, S., Ma, X., and Deoras, A. BASS: Batched attention-optimized speculative sampling. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 8214–8224, 2024.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Ranjan, E. [V1][spec decode] add dynamic SD. vLLM pull request #32374, https://github.com/vllm-project/vllm/pull/32374, 2026.

Sadhukhan, R., Chen, J., Chen, Z., Tiwari, V., Lai, R., Shi, J., Yen, I. E.-H., May, A., Chen, T., and Chen, B. MagicDec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *International Conference on Learning Representations (ICLR)*, 2025.

Saxena, A. Prompt lookup decoding. https://github.com/apoorvumang/prompt-lookup-decoding, 2023.

Su, Q., Giannoula, C., and Pekhimenko, G. The synergy of speculative decoding and batching in serving large language models. *arXiv preprint arXiv:2310.18813*, 2023.

vLLM contributors. Dynamic speculative decoding causes aggregate-throughput collapse under concurrency at the batch-size threshold. vLLM issue #49548, https://github.com/vllm-project/vllm/issues/49548, 2026a.

vLLM contributors. DSD arms pay a large baseline tax vs no-spec under production defaults. vLLM issue #49986, https://github.com/vllm-project/vllm/issues/49986, 2026b.

vLLM contributors. [bugfix][MRV2][spec decode] honor dynamic K in autoregressive speculators. vLLM pull request #57053, https://github.com/vllm-project/vllm/pull/57053, 2026c.

Williams, S., Waterman, A., and Patterson, D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.

Wu, Z., Zhou, Z., Verma, A., Prakash, A., Rus, D., and Low, B. K. H. TETRIS: Optimal draft token selection for batch speculative decoding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33329–33345, 2025.

Xia, H., Yang, Z., Dong, Q., Wang, P., Li, Y., Ge, T., Liu, T., Li, W., and Sui, Z. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

Zhang, Z., Xu, J., Liang, T., Chen, X., He, Z., Wang, R., and Tu, Z. Draft model knows when to stop: Self-verification speculative decoding for long-form generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 16685–16697, 2025.